

1 What is a Regular Expression?

A **regular expression** is a **special string** used to **describe patterns in text**.

Think of it as a **search pattern**. You can use it to:

- Find a word or pattern in a text.
- Validate inputs like emails, phone numbers, URLs.
- Replace parts of text.
- Split strings in complex ways.

Example Conceptually

- Text: "My phone number is 01152063478"
- Regex: `\d{11}` → finds **11 consecutive digits** → "01152063478"

2 Python Module for Regex

Python has a built-in module called `re`:

```
import re
```

Main functions:

Function	Description
<code>re.search()</code>	Searches first occurrence of pattern
<code>re.findall()</code>	Returns all matches as a list
<code>re.match()</code>	Checks if pattern matches from the beginning
<code>re.split()</code>	Splits string using regex as delimiter
<code>re.sub()</code>	Replaces matches with a string

3 Basic Regex Symbols

Here's a cheat sheet for beginners:

Symbol	Meaning
<code>.</code>	Any character except newline
<code>\d</code>	Any digit [0-9]

Symbol	Meaning
\D	Any non-digit
\w	Any word character [a-zA-Z0-9_]
\W	Any non-word character
\s	Any whitespace
\S	Any non-whitespace
^	Start of string
\$	End of string
*	0 or more repetitions
+	1 or more repetitions
?	0 or 1 repetition
{n}	Exactly n repetitions
{n,m}	Between n and m repetitions
[]	Set of characters, e.g., [abc]
\	Escape character

4 Simple Examples

1. Searching a number

```
import re

text = "My number is 01152063478"
pattern = r"\d{11}" # 11 digits
match = re.search(pattern, text)
print(match.group()) # Output: 01152063478
```

2. Find all emails in text

```
text = "Contact: alice@example.com, bob@mail.com"
pattern = r"\w+@\w+\.\w+"
emails = re.findall(pattern, text)
print(emails) # ['alice@example.com', 'bob@mail.com']
```

3. Replace text

```
text = "I love cats"
pattern = r"cats"
new_text = re.sub(pattern, "dogs", text)
print(new_text) # I love dogs
```

4. Split text using regex

```
text = "apple, banana; orange|mango"
pattern = r"[;|,]"
fruits = re.split(pattern, text)
print(fruits) # ['apple', 'banana', 'orange', 'mango']
```

5 Advanced Patterns

1. Optional characters

```
pattern = r"colou?r" # matches "color" or "colour"
```

2. Start & End

```
pattern = r"^Hello" # starts with Hello
pattern = r"world$" # ends with world
```

3. Grouping

```
pattern = r"(\d{3})-(\d{4})"
text = "Call 123-4567"
match = re.search(pattern, text)
print(match.groups()) # ('123', '4567')
```

4. Escape special characters

```
pattern = r"\." # matches a literal dot
```

6 Flags

Regex has **modifiers** called flags:

Flag	Meaning
re.I	Case-insensitive
re.M	Multi-line mode (^ and \$ match each line)
re.S	Dot matches newline (. matches \n)

```
pattern = r"hello"
text = "Hello world"
match = re.search(pattern, text, re.I)
print(match.group()) # Hello
```

7 Tips to Become a Regex Hero

1. Always use **raw strings**: `r"pattern"` → avoids double escaping.
2. Test your regex on **regex101.com**.

3. Start small, build step by step.
 4. Break down complex patterns into **groups**.
 5. Learn **lookaheads**, **lookbehinds**, and **non-capturing groups** later.
-



Quick Example: Validate an Egyptian Phone Number

import re

```
phone = "+20 01152063478"
pattern = r"0\d{10}" # starts with 0 + 10 digits
match = re.search(pattern, phone)
if match:
    print(match.group()) # 01152063478
```

If you want, I can make a **complete Python regex cheat sheet with examples for every case from beginner to expert**, including **emails, URLs, phone numbers, passwords, and complex patterns**.

Do you want me to do that next?